



《AI大模型Ragent项目》——为什么上传30MB文件占了100MB内存？

来自： 拿个offer-开源&项目实战

 马丁

2026年03月25日 23:14

知识库模块需要把用户上传的文档存到 S3 兼容的对象存储（我们用的 RustFS）。功能跑通之后，随手用 VisualVM 看了一眼内存——上传一个 30MB 的 PDF，堆内存直接飙了将近 100MB。
这篇文章完整记录排查过程和最终方案。先说结论：SDK 提供的所有主流上传 API（同步/异步/TransferManager）我们都试了，堆内存都会飙。最终用 S3Presigner 生成预签名 URL + HttpURLConnection 流式上传，VisualVM 下观察不到文件大小级别的堆增长。

看这个问题的背景就是，上一篇文章讲了上传的文件是临时文件不占内存，然后我也用了 S3 的流式 API，为什么还是占用高内存？就有了一步步的测试，如下所示。

问题现场

技术栈：Spring Boot + AWS SDK v2 (2.40.2) + RustFS (S3 兼容)。
上传代码就是官方文档的标准写法：

```
try (InputStream uploadIs = file.getInputStream()) {
    s3Client.putObject(
        PutObjectRequest.builder()
            .bucket(bucketName)
            .key(s3Key)
            .contentType(detectedContentType)
            .build(),
        RequestBody.fromInputStream(uploadIs, size)
    );
}
```

VisualVM 观察：上传 30MB 文件，堆内存瞬间飙升约 100MB，反复测试稳定复现。

这里说的是增量。应用空闲时堆稳定在某个水平，上传时突然跳约 100MB，GC 后回落。

定位过程

上传逻辑涉及 Tika 类型检测、创建临时文件、调用 SDK 上传几个步骤。哪一步吃的内存？我用了个简单办法——在不同位置插 if(true) return null;，相当于对代码做二分查找。

第一刀：Tika 检测之后直接返回

```
String detectedContentType = TIKA.detect(is, originalFilename);
if (true) return null;
```

首次调用涨了约 30MB（Tika 初始化加载 MIME 数据库，一次性开销），后续调用无波动。排除。

第二刀：临时文件创建之后直接返回——无波动，排除。

第三刀：putObject 调用之后返回——飙升约 100MB。就是 putObject 的问题，接下来开始排查。

六次尝试，全部失败

确定问题在 putObject 之后，很自然地想换一种方式调 SDK。以下是依次尝试的方案。

1. RequestBody.fromFile() 替代 fromInputStream()

直觉上 fromFile() 应该走 FileChannel，流式读取不需要全量缓冲。

```
s3Client.putObject(request, RequestBody.fromFile(tempFile));
```

结果：~90MB。SDK 内部照样把文件读进了堆。

2. 开启 chunked encoding

S3Configuration 上有个 chunkedEncodingEnabled(true) 选项，理论上分块发送避免一次性缓冲。
结果：从 100MB 降到约 50MB，改善了但没根治。

3. 显式指定 Apache HTTP 客户端

SDK v2 的 s3 模块默认同步客户端就是 ApacheHttpClient。显式配置它，配合 chunked encoding 试试：

```
<dependency>
  <groupId>software.amazon.awssdk</groupId>
  <artifactId>apache-client</artifactId>
</dependency>
```

结果：约 50–60MB，和尝试二差不多。HTTP 客户端层面不是瓶颈。

4. S3TransferManager

AWS 官方推荐的高级传输工具，支持自动分片：

```
S3TransferManager transferManager = S3TransferManager.builder()
    .s3Client(asyncClient)
    .build();
transferManager.uploadFile(UploadFileRequest.builder()
    .putObjectRequest(request)
    .source(tempFile)
    .build());
```

结果：~90MB。

5. S3AsyncClient + AsyncRequestBody.fromFile()

```
s3AsyncClient.putObject(request, AsyncRequestBody.fromFile(tempFile)).join();
```

结果：~90MB。异步客户端换了线程模型，签名流程没变。

6. RustFS 官方示例

抱着最后一丝希望，直接跑 RustFS 官方 Java 示例：

```
s3.putObject(req, Paths.get("hello.txt"));
```

结果：~90MB。
到这里基本可以确认：不是某个 API 的问题，是 SDK 上传管线的整体行为。

这些具体的 MB 可能没那么准确，因为同时项目有一些后台任务，但是整体预计偏差不会超过 10-30 MB。

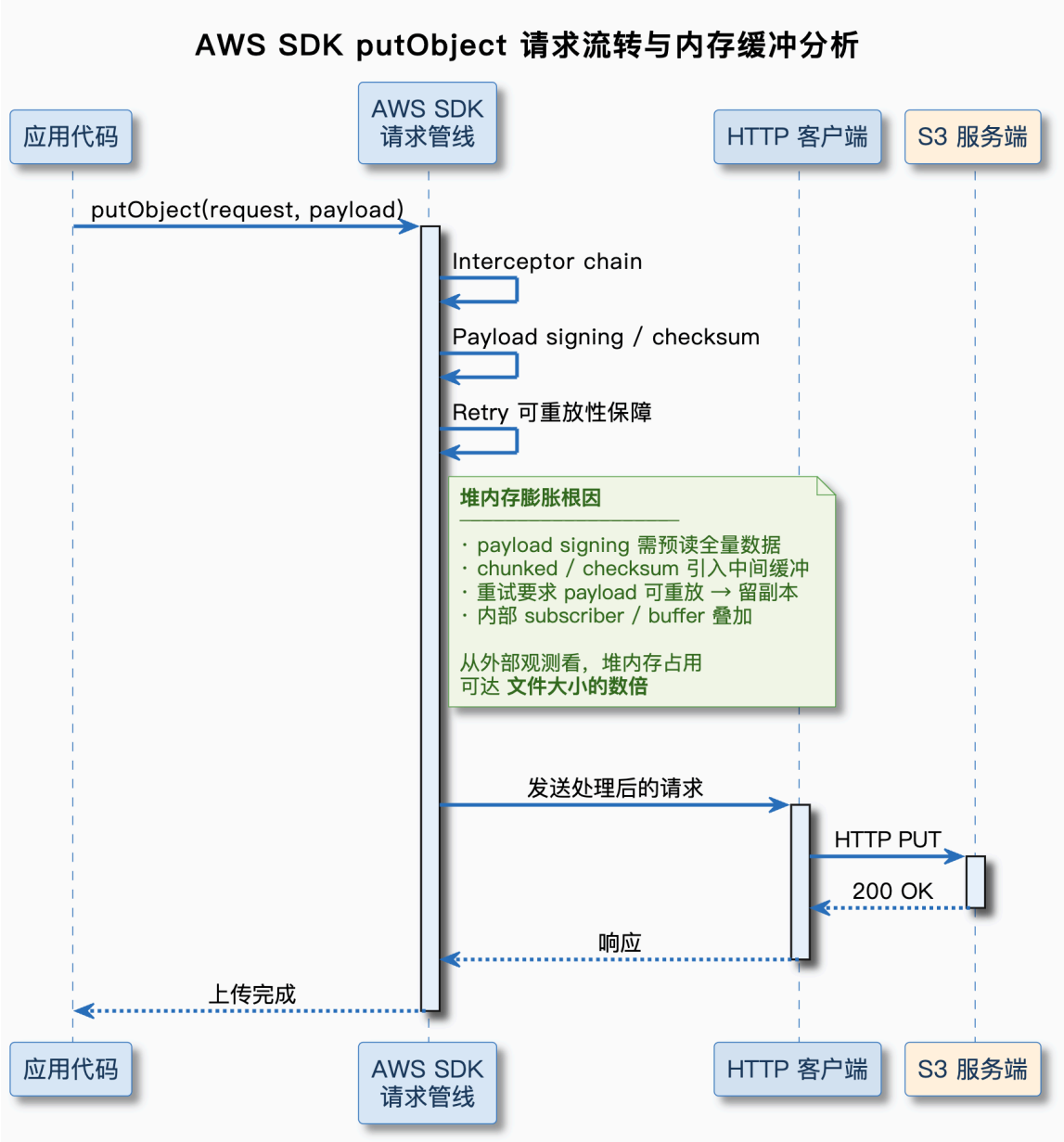
方案对比

方案	堆内存增量
putObject + fromInputStream	~100MB
putObject + fromFile	~90MB
putObject + fromFile + chunkedEncoding + Apache 客户端	~50MB
S3TransferManager	~90MB

S3AsyncClient + AsyncRequestBody.fromFile	~90MB
RustFS 官方示例 putObject(req, Path)	~90MB
S3Presigner + HttpURLConnection	近似 0MB

根因分析

要理解为什么所有 putObject 变体都飙内存，得看 SDK 请求管线里发生了什么。
上传管线大致经过这些阶段：



导致缓冲的几个因素：

- 1. Payload signing**：SigV4 协议在 HTTPS 下可以走 UNSIGNED-PAYLOAD 跳过 payload 哈希，但在 HTTP 下（内网部署 RustFS 很常见）会进入 payload signing 路径，需要计算整个文件的 SHA-256。开了 chunked encoding 后从 ~100MB 降到 ~50MB，说明 payload signing 确实是原因之一，但不是全部。
- 2. CRC32 校验和**：Java SDK 2.30.0 以后默认会为上传自动计算 CRC32 校验和（x-amz-checksum-crc32），这也需要读取整个文件。这是内存占用的另一个重要因素。
- 3. Interceptor chain**：SDK 内部拦截器链在请求发送前会对 RequestBody 做多次读取和包装，中间操作可能导致数据在堆中驻留。
- 4. Retry 可重放性**：对 fromInputStream() 来说，SDK 需要缓冲数据才能在失败后重试。但 fromFile() 的 ContentStreamProvider 每次调用 newStream() 都会重新打开文件，天然支持重放——所以 retry 机制解释不了 fromFile() 的 ~90MB 飙升，缓冲的主要来源在管线的其他环节。
- 5. 内部 subscriber / buffer**：异步路径中，SDK 的 subscriber 自身会做 buffering。

这些因素叠加起来，堆内存可以飙到文件大小的数倍。我们没有逐行 trace SDK 源码确认每个环节的具体贡献量，但实测结果已经足够说明问题。

1. 这算 SDK 的 bug 吗？

看怎么理解。
从用户角度看，算。SDK 提供了 fromFile()、fromInputStream() 这些 API，表示支持流式上传，但跑起来整个文件都被缓冲到堆里了，API 语义和实际行为不一致。GitHub 上 [#5647](#) 曾被 AWS 团队标记为 bug (p2)，后来以 Closed as not planned 关闭——承认了问题，但不打算在现有架构下修。
从架构角度看，签名管线设计如此，要改动面太大。而我们的 Presigner 方案就是完全绕开这条管线，代价是失去了 SDK 内置的自动重试。

2. 升级到最新版能解决吗？

不能。从 2.40.2 到最新的 2.42.x，这块架构没有变化：

Issue	状态	描述
#5647	Closed as not planned	S3AsyncClient putObject 内存飙升，100MB 文件占 400MB 堆
#5859	Open	v2.30 起 InputStream 行为变更导致流泄漏
aws-crt-java#894	Open	CRT 客户端也需要高内存

AWS 文档建议的方法我们全试了：提供 contentLength → 仍然缓冲；用 fromFile() → ~90MB；用 AsyncClient + multipart → 更差；用 CRT 客户端 → 内存更高。

3. 那 SDK 的流式 API 还有什么用？

SDK 的流式 API 解决的不是内存问题，是编程接口问题。
没有流式 API，你只能这样写：

```
byte[] data = Files.readAllBytes(path); // 业务层一份
s3Client.putObject(request, RequestBody.fromBytes(data)); // SDK 内部再来一份
// 堆里同时两份完整文件
```

有了流式 API：

```
s3Client.putObject(request, RequestBody.fromFile(path));
// 业务层不持有完整文件，只有 SDK 内部那一份缓冲
```

方式	业务层堆占用	SDK 内部	总占用
fromBytes(Files.readAllBytes())	1 份 byte[]	额外缓冲	更高
fromFile(path)	0 (磁盘)	额外缓冲	仅 SDK 管线缓冲
fromInputStream(is, size)	0 (流)	额外缓冲	仅 SDK 管线缓冲

SDK 内部那份缓冲甩不掉，但至少业务层不用再多一份。流式 API 的价值是少一份拷贝，不是零内存。大多数场景（小文件、内存充裕、偶发上传）够用了，只有大文件 + 低内存 + 高并发才需要绕开管线。

4. 签名机制深入：为什么 putObject 必须缓冲文件

要彻底理解内存问题的根源，需要了解 AWS S3 的签名机制。

4.1 AWS SigV4 签名算法

AWS S3 使用 Signature Version 4 (SigV4) 签名算法来验证请求的合法性：

```
Signature = HMAC-SHA256(SigningKey, StringToSign)

StringToSign =
```

```
Algorithm + "\n" +
RequestDateTime + "\n" +
CredentialScope + "\n" +
HashedCanonicalRequest

HashedCanonicalRequest = SHA256(
    HTTPMethod + "\n" +
    CanonicalURI + "\n" +
    CanonicalQueryString + "\n" +
    CanonicalHeaders + "\n" +
    SignedHeaders + "\n" +
    SHA256(RequestBody) // ← 关键：默认需要计算请求体的 SHA-256 哈希
)
```

这是 SDK 默认实现的内存问题根源之一：要计算 `SHA256(RequestBody)`，SDK 需要先读取整个请求体。对于不可回溯的 `InputStream`，SDK 选择了全部缓冲到内存的策略。

AWS 协议层面的绕过方案：

AWS 也意识到了这个问题，提供了两种绕过方案：

```
# 方案一：跳过 payload 签名
x-amz-content-sha256: UNSIGNED-PAYLOAD

# 方案二：分块签名 (chunked signing)
x-amz-content-sha256: STREAMING-AWS4-HMAC-SHA256-PAYLOAD
```

`UNSIGNED-PAYLOAD`：签名时使用固定字符串，不计算文件哈希（预签名 URL 使用的就是这个）

分块签名：每个 chunk 单独签名，支持流式传输

所以不是协议层面必须读整个文件，而是 **SDK 默认实现选择了最保守的方式**。

4.2 为什么需要签名请求体？

签名请求体是为了保证数据完整性和防止中间人攻击：

- 1. **防篡改**：签名包含请求体哈希，服务端可以验证内容是否被篡改
- 2. **防重放**：结合时间戳，防止请求被重放攻击
- 3. **身份验证**：证明请求来自持有密钥的合法用户

这是一个安全性和性能的权衡：更强的安全保障，代价是必须缓冲整个文件。

4.3 预签名 URL：用 `UNSIGNED-PAYLOAD` 绕过请求体签名

预签名 URL 的核心思路是：**把 SigV4 签名从 Authorization header 换成 query string，并使用 `UNSIGNED-PAYLOAD`**。这不是另一种签名算法，而是同一套 SigV4 的 query-string 变体（AWS 官方称为 "Using Query Parameters (AWS Signature Version 4)"）。

传统方式 (SDK putObject)：

- 1. 读取文件 → 计算 `SHA-256` (或 `CRC32`) → 生成签名 → 上传文件
 - ↑ 签名/校验/重试/内部 buffering 共同导致内存占用

预签名 URL 方式：

- 1. 生成签名 (使用 `UNSIGNED-PAYLOAD`，不需要文件) → 签名编码到 URL query string
- 2. 上传文件 (直接流式传输)
 - ↑ 签名和传输完全分离

预签名 URL 在签名时使用 `UNSIGNED-PAYLOAD` 标记：

```
HashedCanonicalRequest = SHA256(
    HTTPMethod + "\n" +
    CanonicalURI + "\n" +
```

```
CanonicalQueryString + "\n" +  
CanonicalHeaders + "\n" +  
SignedHeaders + "\n" +  
"UNSIGNED-PAYLOAD" // ← 不计算请求体哈希，用固定字符串  
)
```

签名信息编码到 URL 的查询参数中：

```
https://bucket.s3.amazonaws.com/key?  
X-Amz-Algorithm=AWS4-HMAC-SHA256&  
X-Amz-Credential=...&  
X-Amz-Date=20260325T140000Z&  
X-Amz-Expires=600&  
X-Amz-Signature=abc123...&  
X-Amz-SignedHeaders=host
```

这样，签名计算时完全不需要读取文件，上传时用 `URLConnection` 直接流式发送，全程不缓冲。

预签名 URL 绑定的内容：

- HTTP 方法（PUT/GET/DELETE）
- Bucket + Key（资源路径）
- Region + Credential Scope
- 过期时间（最长 7 天，对长期 IAM 凭证；STS/role 临时凭证会提前失效）
- 所有被签入的 headers（如 `Content-Type`，实际上传时必须精确匹配）

安全性权衡：

- ✅ 仍然验证身份（签名包含凭证）
 - ✅ 仍然防重放（签名包含时间戳）
 - ✅ 限制到特定方法/路径/时效/签名头集合
 - ⚠️ 不验证请求体完整性（UNSIGNED-PAYLOAD）
 - ⚠️ **预签名 URL 本质上是 bearer token**：谁拿到谁就能在有效期内使用，且可以被重复使用（不是一次性票据）
- 传输过程中的机密性和抗链篡改主要依赖 TLS
- 若需要对象内容级完整性校验，可额外签入 `x-amz-checksum-*` 头，让 S3 在服务端校验

4.4 阿里云 OSS：签名设计对流式上传更友好

之前我都是用阿里云的 OSS，今天拿 OSS v2 版本的 SDK 跑了下，结果发现是没有问题的。相对于咱们实现的方案，重试方案内部集成了。

对比一下阿里云 OSS 的签名算法：

```
Signature = Base64(HMAC-SHA1(AccessKeySecret, StringToSign))  
  
StringToSign =  
    HTTP-Verb + "\n" +  
    Content-MD5 + "\n" +  
    Content-Type + "\n" +  
    Date + "\n" +  
    CanonicalizedOSSHeaders +  
    CanonicalizedResource
```

关键区别： OSS 的签名算法（`StringToSign`）不包含请求体内容，签名阶段完全不需要读取文件。

而 AWS SigV4 默认要求在签名中包含 `SHA256(Payload)`，SDK 需要在签名阶段先读取文件计算哈希，再读取文件进行传输——对于不可回溯的 `InputStream`，SDK 选择了全部缓冲到内存的策略。

当然，AWS 协议层面也提供了 `UNSIGNED-PAYLOAD` 等机制来绕过这个限制（预签名 URL 正是利用了这一点），但 SDK 默认没有启用，这也是我们需要手动优化的原因。

这种签名设计上的差异，加上 `Content-Length` 计算、重试缓冲等 SDK 实现层面的因素，共同导致了 AWS SDK 默认上传时的内存问题。

OSS 签名只需要：

HTTP 方法 (PUT/GET/DELETE)

Content-MD5 (可选, 用于服务端校验, 不是必须的)

Content-Type

Date (时间戳)

自定义 OSS 头 (x-oss-*)

资源路径 (bucket + key)

关于 Content-MD5:

Content-MD5 是**可选字段**, 不提供时签名计算中为空字符串

即使提供了 Content-MD5, 也不需要**在签名阶段**读取文件

哈希算法都可以流式计算 (MD5、SHA-256 都支持):

```
// MD5 流式计算示例
MessageDigest md5 = MessageDigest.getInstance("MD5");
byte[] buffer = new byte[8192];
int bytesRead;
while ((bytesRead = inputStream.read(buffer)) != -1) {
    md5.update(buffer, 0, bytesRead); // 边读边更新哈希状态
}
byte[] hash = md5.digest(); // 最终计算结果

// SHA-256 同样可以流式计算
MessageDigest sha256 = MessageDigest.getInstance("SHA-256");
while ((bytesRead = inputStream.read(buffer)) != -1) {
    sha256.update(buffer, 0, bytesRead); // 边读边算
}
byte[] sha256Hash = sha256.digest();

// 内存占用: 只有 8KB buffer, 与文件大小无关
```

与 AWS 的关键区别:

AWS SDK 的问题: 不是 SHA-256 不能流式计算, 而是**算完哈希后流已经读完了, 还要再读一次来发送数据**。对于不可回溯的 InputStream, SDK 只能全部缓冲到内存

OSS 的优势: Content-MD5 是可选的, 签名阶段不需要, 可以在上传过程中边读边算, 或者直接不提供

没有 SHA256(RequestBody) 这样的强制要求, 签名计算时完全不需要读取文件。再加上 OSS SDK 在 Content-Length 处理和流的消费上做了更好的封装, 这就是为什么 OSS v2 SDK 可以直接 `BinaryData.fromStream()` 流式上传, 而 AWS SDK 默认实现会缓冲整个文件, 需要我们通过预签名 URL 等方式手动绕过。

OSS SDK 的上传流程:

```
// 1. 构建 StringToSign (不包含请求体)
String stringToSign =
    "PUT\n" +
    "\n" +
    "application/pdf\n" +
    "Wed, 25 Mar 2026 14:00:00 GMT\n" +
    "/bucket/key";

// 2. 计算签名 (纯 CPU, 无 IO)
String signature = Base64.encode(HMAC_SHA1(secret, stringToSign));

// 3. 设置 Authorization 头
request.setHeader("Authorization", "OSS " + keyId + ":" + signature);

// 4. 流式发送请求体 (签名已完成)
request.setEntity(new InputStreamEntity(inputStream, size));
```

整个过程中，InputStream 完全没有被读取。OSS SDK 使用 Apache HttpClient 的 InputStreamEntity 实现流式传输，内存占用只有 4-8KB 的 buffer。

4.5 三种方案对比总结

维度	AWS SDK putObject	AWS 预签名 URL	阿里云 OSS SDK
签名算法	SigV4 (SHA-256)	SigV4 (UNSIGNED-PAYLOAD)	OSS (SHA-1)
请求体参与签名	✔ 必须计算哈希	✘ 不参与	✘ 不参与
内存占用	文件大小的 3-4 倍	~8KB buffer	~4-8KB buffer
流式上传	✘ 假流式（内部缓冲）	✔ 真流式	✔ 真流式
自动重试	✔ SDK 内置	✘ 需业务层实现	✔ SDK 内置
代码复杂度	简单	中等	简单
安全性	最高（签名请求体）	中等（HTTPS 保护）	中等（HTTPS 保护）

适用场景：

AWS SDK putObject：

- ✔ 小文件（< 10MB）
- ✔ 内存充裕
- ✔ 需要最强安全保障
- ✘ 大文件 + 高并发

AWS 预签名 URL：

- ✔ 大文件（> 50MB）
- ✔ 高并发场景
- ✔ 内存受限环境
- ⚠ 需要自己实现重试逻辑
- ⚠ 接受 UNSIGNED-PAYLOAD 的安全权衡

阿里云 OSS SDK：

- ✔ 所有场景（签名算法天然支持流式）
- ✔ 大文件 + 高并发
- ✔ 自动重试
- ✔ 代码简单

本质区别：

AWS SigV4 签名算法**设计上需要请求体哈希**，导致必须缓冲文件

阿里云 OSS 签名算法**设计上不包含请求体**，天然支持流式上传

AWS 预签名 URL 通过 UNSIGNED-PAYLOAD 绕过请求体签名，实现流式上传

这就是为什么我们在使用 AWS S3（或兼容服务如 RustFS）时，需要用预签名 URL 方案来优化内存占用。

5. 常见说法 vs 实测

网上很多分析是看文档推理，没跑过：

常见说法	实测结果
用 File 上传，SDK 直接从磁盘读，不加载到内存	fromFile() 30MB 文件 ~90MB 堆飙升
fromInputStream 指定长度就能避免全量加载	指定 contentLength 后仍 ~100MB
分段上传只存几个分块	30MB 文件用 TransferManager ~90MB
改 HTTPS 就能用 UNSIGNED-PAYLOAD 解决	#5647 在 AWS S3 + HTTPS 下同样复现

预签名解释

类比：寄快递，假设你要寄一个 500 斤的大箱子。

方式一：AWS SDK（普通寄件）

快递员说：我要先验货、称重、拍照，然后才能寄。

流程：

- 1. 快递员把你的 500 斤箱子搬到他的车上（占用车的空间）
- 2. 在车上打开箱子，逐件检查、称重、拍照
- 3. 检查完毕，填写单据、贴标签
- 4. 最后才运走

问题：快递员的车必须腾出 500 斤的空间来放你的箱子，才能完成检查。

对应到技术上：AWS SDK 必须把整个文件加载到堆内存中，才能完成签名计算。

方式二：预签名 URL

你提前打电话给快递公司：我要寄一个 500 斤的箱子到北京，帮我把手续先办好。

快递公司远程给你开了一张**预授权单据**（预签名 URL），上面写着：

- 收件地址：北京
- 重量：500 斤
- 授权码：ABC123
- 有效期：10 分钟

流程：

- 1. 快递员上门，核对预授权单据
- 2. 直接把箱子搬上车就走，**不需要打开检查**
- 3. 箱子全程密封：你家 → 快递车 → 中转仓 → 北京

优势：快递员的车不需要额外空间来拆箱检查，只要能装下箱子就行。

对应到技术上：签名提前算好放在 URL 里，上传时文件直接流式传输，不需要在内存中缓冲。

概念对照表

生活例子	技术概念
箱子（500 斤）	文件（500MB）
快递员的车	服务器堆内存
拆箱检查验货	计算 SHA-256 签名
预授权单据	预签名 URL
密封直接搬走	流式传输（8KB buffer）

数据流对比

AWS SDK 方式：

文件 → 整个加载到内存(500MB) → 计算签名 → 上传

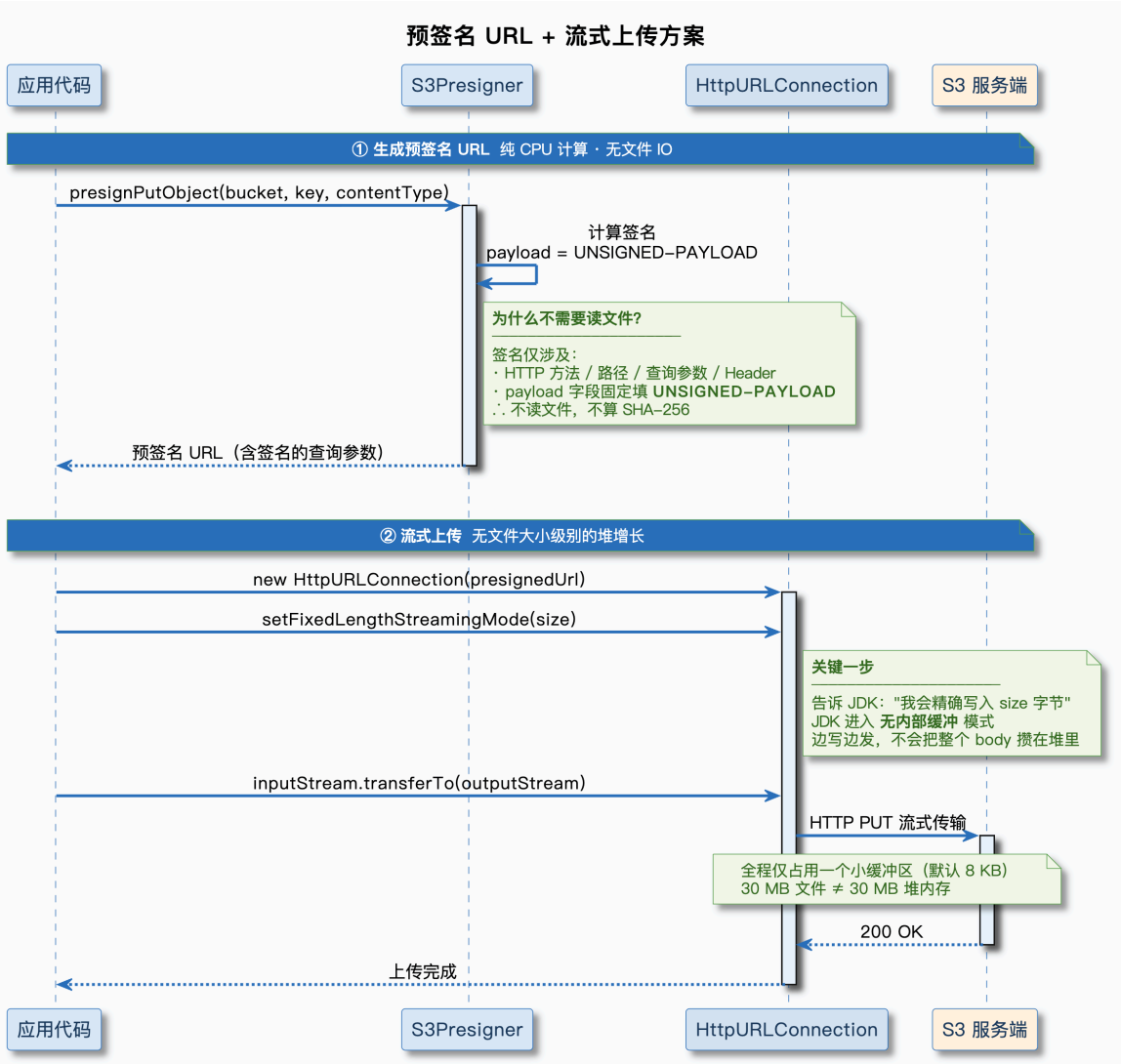
预签名 URL 方式：

提前计算签名(不需要文件) → 文件 → 直接流式上传(只需 8KB buffer)

一句话总结：AWS SDK 必须先把整个文件搬进内存里检查，才能上传；预签名 URL 提前把手续办好，文件直接流过去，不在内存里停留。

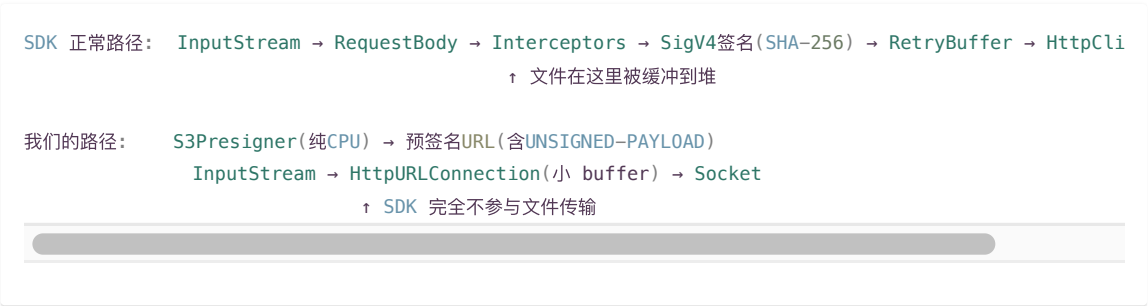
解决方案：预签名 URL + HttpURLConnection

预签名 URL 的签名方式完全不同：



1. 为什么能绕过所有问题

核心就是把签名和传输拆开了，文件数据从头到尾不经过 SDK：



两个关键点：

- 1. **签名不涉及 payload：**预签名 URL 把鉴权信息编码到 URL 查询参数中，payload 标记为 UNSIGNED-PAYLOAD，不需要读文件内容，不需要算 SHA-256。
- 2. **JDK 原生流式传输：**`setFixedLengthStreamingMode(size)` 告诉 JDK 精确的 Content-Length，JDK 直接把写入转发到 Socket，不做内部缓冲。`transferTo` 内部用固定大小 buffer 中转（JDK 9-17 为 8KB，18+ 为 16KB），全程堆增量近似为零。

2. 完整实现

2.1 Spring 配置类

```
@Configuration
public class RestFSS3Config {

    @Bean
    public S3Client s3Client(@Value("${rustfs.url}") String rustfsUrl,
                             @Value("${rustfs.access-key-id}") String accessKeyId,
                             @Value("${rustfs.secret-access-key}") String secretAccessKey) {
        return S3Client.builder()
            .endpointOverride(URI.create(rustfsUrl))
            .region(Region.US_EAST_1)
            .credentialsProvider(
                StaticCredentialsProvider.create(
                    AwsBasicCredentials.create(accessKeyId, secretAccessKey)))
            .forcePathStyle(true)
            .build();
    }

    /**
     * S3 预签名器：签名在 URL query 参数中完成，
     * 配合 HttpURLConnection 实现零堆内存的流式文件上传
     */
    @Bean
    public S3Presigner s3Presigner(@Value("${rustfs.url}") String rustfsUrl,
                                     @Value("${rustfs.access-key-id}") String accessKeyId,
                                     @Value("${rustfs.secret-access-key}") String secretAccessKey) {
        return S3Presigner.builder()
            .endpointOverride(URI.create(rustfsUrl))
            .region(Region.US_EAST_1)
            .credentialsProvider(
                StaticCredentialsProvider.create(
                    AwsBasicCredentials.create(accessKeyId, secretAccessKey)))
            .serviceConfiguration(S3Configuration.builder()
                .pathStyleAccessEnabled(true)
                .build())
            .build();
    }
}
```

2.2 上传服务核心代码

代码在这里，我就不贴了，大家按需阅读：S3FileStorageService#upload，整体代码逻辑很绕，我认为知道概念即可，不用深入学习。

3. 首次调用的初始化开销

首次上传时堆内存有约 50MB 的一次性增长：Tika 加载 MIME 数据库约 30MB，S3Presigner 和 HttpURLConnection 的类加载 / JIT 约 20MB。后续上传观察不到持续性堆增长。如果对首次延迟敏感（比如容器冷启动），可以启动时做一次空文件上传预热。

注意事项

安全：使用预签名 URL + UNSIGNED-PAYLOAD 时，请求体不参与 SigV4 签名；传输过程中的机密性和抗链路篡改主要依赖 TLS。

自动重试：绕开了 SDK 管线也意味着失去内置重试，上传失败需要业务层自己处理。

文件大小限制：setFixedLengthStreamingMode 接受 long，理论上没问题。如果要做其他业务超大文件上传，比如 5GB 时，需要考虑使用分块上传。

并发场景：用 SDK 的 putObject，N 个并发上传 30MB 文件，峰值堆增量约 $N \times 30\sim 100\text{MB}$ ，10 个并发就是 1GB。用 Presigner 方案，并发时额外堆占用只有 transferTo 的小 buffer 和连接对象，远小于 SDK 路径。这才是这个优化在生产环境最大的价值。

S3Client 还需要吗？需要。S3Presigner 只能签名，不能执行 getObject、deleteObject 等操作。读取和删除仍然走 S3Client，但 these 操作请求体很小或没有请求体，不存在内存问题。getObject 返回的 ResponseInputStream 本身就是流式的，只要不调 readAllBytes() 就不会有问题。

小结

最大的教训：不要假设 SDK 内部行为符合直觉。`fromFile()` 看起来应该流式读取，`AsyncRequestBody` 看起来应该异步流式传输，但实际上都被签名管线截胡了。

为什么网上几乎没人聊这个问题？你去搜相关资料会发现，涉及 AWS SDK 上传内存问题的文章非常少。这不是因为问题不存在，而是因为它太难被注意到了：

1. **文件不够大**：绝大多数业务场景处理的都是几十到几百 MB 的文件，内存波动不痛不痒
2. **内存给得够多**：企业级应用的 JVM 堆动辄 8~16GB 甚至更高，复杂业务的后管服务再部上 2~3 个节点，上传时多占个几百 MB 根本看不出来
3. **上传频率不高**：文件上传在大多数系统里不是热点路径，偶尔飙一下很快就被 GC 回收了，监控面板上连个毛刺都算不上
4. **云厂商 SDK 屏蔽了细节**：不少团队用的是阿里云 OSS、腾讯云 COS 这类厂商封装好的客户端，底层实现和 AWS SDK 不同，未必有同样的问题

说白了，这几个条件叠在一起，99% 的开发者都不会碰到——或者碰到了也不会意识到。

我也是巧了。上一篇在研究文件解析时顺手开着 VisualVM，恰好看到了上传阶段的内存跳变，顺藤摸瓜才有了这篇文章。**如果不是刚好在观察内存，这个问题我自己也会直接忽略过去。**

所以与其说这篇文章的价值在于解决了一个 SDK 的内存问题，不如说它是一个提醒：**做功能的时候，多瞄一眼内存和 CPU 的变化。**很多生产环境的故障，不是突然爆发的，而是一直在那里，只是没人看。

参考资料

[AWS SDK for Java v2 – S3 Stream Upload Best Practices](#)

[GitHub Issue #5647 – S3AsyncClient.putObject memory issues](#)

[GitHub Issue #5859 – AWS v2.30 SDK InputStream behavior changes](#)

[GitHub Issue #894 – CRT client low memory requirement](#)

[AWS payload_signing_enabled setting](#)

[GitHub Issue #4721 – ENABLE_PAYLOAD_SIGNING deprecated](#)

[SigV4 Signature Calculations](#)

[S3 Presigned URL Upload with Checksum](#)

[Work with Amazon S3 pre-signed URLs](#)

[RequestBody Javadoc \(AWS SDK for Java 2.x\)](#)